Open Access | https://doi.org/10.25080/uekx6559



SciPy 2026

July 13 - July 19, 2026

Proceedings of the 25th Python in Science Conference
ISSN: 2575-9752

NODEFit: Fit time-series data with a Neural Differential Equation

Pavan B. Govindaraju[1]

[1]SplitFXM

Abstract

Time-series data often arise from underlying physical, biological, and engineering systems governed by continuous dynamics. Standard discrete-time regressors, models that predict the next sample from a finite history of past observations, and fixed-form curve fits can interpolate measurements yet fail to extrapolate or to represent stochastic structure when the governing equations are unknown. Neural Ordinary Differential Equations (Neural ODEs) learn a continuous-time evolution law with a neural network, and Neural Stochastic Differential Equations (Neural SDEs) extend this idea to systems driven by random noise. This paper introduces NODEFit, an open-source Python package that wraps differentiable PyTorch solvers (`torchdiffeq` for ODEs and `torchsde` for SDEs) to fit Neural ODEs and Neural SDEs to measured data. Relative to calling those libraries directly, NODEFit lets a user specify a drift network, and for SDEs a diffusion network, once and reuse that architecture across every series in a dataset, while the package manages joint training of the corresponding parameters. That bookkeeping is especially useful for bulk processing of large collections of series, such as the PTB-XL cohort study in this paper. We motivate the approach with examples where template-based and discrete-time methods do not perform well, recover underlying dynamics on synthetic benchmarks, reconstruct the classical Lotka–Volterra predator–prey model, and evaluate the method on clinical electrocardiogram (ECG) recordings from PTB-XL, including stochastic heart-rate dynamics in arrhythmias such as atrial fibrillation across nearly 2,000 patients. Finally, we summarize when practitioners should prefer continuous-time neural ODE and SDE models.

Keywords neural-ode, neural-sde, time-series, machine-learning, differential-equations

Published Jul 13, 2026

Correspondence to
Pavan B. Govindaraju
pavan@splitfxm.com

Open Access

Copyright © 2026 Govindaraju. This is an open-access article distributed under the terms of the Creative Commons Attribution 4.0 International license, which enables reusers to distribute, remix, adapt, and build upon the material in any medium or format, so long as attribution is given to the creator.

1. Introduction

Physical phenomena are commonly governed by differential equations that describe how a system evolves continuously in time. Traditional time-series methods often rely on discrete-time models: predictors that map a finite history of samples $(y_{t-k}, \ldots, y_t)$ to a next-step value y_{t+1}, without an explicit notion of rates of change between observation times. A concrete failure mode is irregular sampling. An autoregressive or recurrent model trained at a fixed step Δt has no well-defined state between grid points. Querying it at an intermediate time, or after a gap of several Δt, requires ad hoc interpolation or multi-step rollouts that accumulate error. Continuous-time neural models were introduced in part to address exactly this setting [1], [2].

An ordinary differential equation (ODE) instead specifies a vector field $dy/dt = f(y, t)$ whose integral yields the trajectory. Neural Ordinary Differential Equations (Neural ODEs) [1], [3] replace the unknown f with a neural network f_θ and learn the dynamics by matching the integrated trajectory to data.

When the evolution itself is noisy, the natural continuous-time description is a stochastic

differential equation (SDE)[1]: an equation of the form $dy = f(y,t)\,dt + g(y,t)\,dW_t$ in which a drift f is accompanied by a diffusion g driven by Brownian motion W_t. Neural Stochastic Differential Equations (Neural SDEs) [4], [5] learn both f_θ and g_θ from observations.

Consider a concrete motivating example where two state variables rise toward distinct steady-state values according to smooth exponential kinetics, and each measurement is corrupted by noise. One might try nonlinear least squares with `scipy.optimize.curve_fit` [6], specifying a candidate functional form such as a single exponential or logistic. That approach works only when the chosen template matches the true dynamics.

These failures share a root cause: the models describe values at sampled times rather than the rates of change that generated the trajectory. Neural ODEs and Neural SDEs take the opposite view. Instead of fitting $y(t)$ directly, they learn a vector field $f_\theta(y,t)$, and, when needed, a diffusion term $g_\theta(y,t)$, such that integrating forward reproduces the observations. Neural SDEs are particularly useful for noisy observations, where they separate drift from diffusion, yielding mean trajectories and uncertainty bands that envelop the data.

These models have direct applications wherever observations are generated by evolving state variables. In ecology, prey and predator populations are coupled through the Lotka–Volterra equations. In epidemiology, compartment models describe infections and recoveries, and in chemical kinetics, reaction concentrations evolve according to mass-action laws. Related examples also occur in robotics and control, where a vehicle's position and velocity follow continuous dynamics, and in finance, where continuous-time stochastic models describe quantities such as asset prices and interest rates. In each case, measurements may be sparse, irregular, noisy, or incomplete, making it useful to learn a continuous evolution law rather than only a one-step predictor.

NODEFit, built on PyTorch, offers a distinct approach to time-series modeling and addresses a different set of assumptions than many established alternatives. Classical ARIMA models [7] are effective statistical models for regularly sampled series with linear dependence and carefully specified differencing and seasonal structure. Transformer models [8] use self-attention to learn long-range relationships in sequences and can be highly expressive, but they generally treat the sampling grid as part of the representation rather than learning an explicit continuous-time evolution law. Path signatures [9] summarize a time series through iterated integrals and provide useful features for learning from sequential or irregularly sampled paths, but they do not by themselves specify a forward dynamical or stochastic model. NODEFit complements these approaches by learning a vector field, and optionally a diffusion, that can be queried at arbitrary times and used for continuous-time extrapolation.

PySINDy [10] recovers ODEs by sparse regression onto a user-specified library of candidate terms [11]. As with `curve_fit`, that library is a predetermined basis, and if the true vector field is not in it, identification fails. NODEFit, built on PyTorch, instead parameterizes f_θ and g_θ with neural networks, so the form need not be known in advance. Differentiable solvers already exist [3], [4], and, to the author's knowledge, NODEFit is the first high-level Python package that wraps those libraries (`torchdiffeq` and `torchsde`) to fit both Neural ODEs and Neural SDEs to measured time series without such a candidate library.

NODEFit packages these ideas for practitioners. After installing the package, a user passes time-series data, specifies compact drift and diffusion networks, and calls `train` and

[1] Stochastic differential equations and Itô's lemma originate with the Japanese mathematician Kiyosi Itô (1915–2008), who in the 1940s developed a rigorous stochastic integral and, in 1951, the change-of-variable formula now known as Itô's lemma, the stochastic counterpart of the chain rule. These tools later became foundational in mathematical finance: Fischer Black, Myron Scholes, and Robert Merton used them in the early 1970s to derive the Black–Scholes–Merton model for option pricing. Itô spent much of his career at Kyoto University and is widely regarded as the founder of modern stochastic analysis.

`extrapolate`. The package owns the training loop, observation batching, and coordinated updates of drift and diffusion parameters, so the user does not assemble adjoint code, Brownian-tree bookkeeping, or separate optimizer groups. That interface is what distinguishes NODEFit from calling `torchdiffeq` or `torchsde` directly. Benchmarks in this paper show that the resulting continuous-time fits recover the underlying kinetics and extrapolate reliably on the motivating example above, which polynomial, template-based, and discrete-time alternatives struggle to match without prior knowledge of the governing equations.

The remainder of the paper is organized as follows. Methods defines the continuous-time and stochastic models, training objective, and adjoint backpropagation. Implementation describes the solver dependencies, memory-saving methods, installation, and example API. Results evaluates NODEFit's PyTorch-based Neural ODEs and SDEs on deterministic and stochastic benchmarks, including a reconstruction of the classical Lotka–Volterra population model, recovery of noisy dynamics, and clinical case and cohort studies modeling cardiac arrhythmias on PTB-XL ECG data[2] (see Section 15). The Appendix provides additional background on the stochastic calculus used by the SDE adjoint.

2. Methods

2.1. Continuous-time model

We model the latent state evolution as:

$$\frac{dy(t)}{dt} = f_\theta(y(t), t) \tag{1}$$

where $y(t) \in \mathbb{R}^d$ denotes the system state vector at continuous time t, and f_θ is a neural network parameterized by weights θ that approximates the unknown continuous vector field (governing dynamics).

2.2. Stochastic dynamics

For noisy systems, NODEFit supports Neural SDEs [4], [5]:

$$dy = f_\theta(y, t)dt + g_\theta(y, t)dW_t \tag{2}$$

where y is the state vector, t is continuous time, $f_\theta(y, t)$ is the drift neural network capturing deterministic trends, $g_\theta(y, t)$ is a neural network parameterizing the diffusion (state-dependent noise amplitude), and W_t represents a standard Wiener process (Brownian motion).

2.3. Training Objective

Given observations (t_i, y_i), NODEFit minimizes the mean squared error:

$$\mathcal{L} = \sum_i | \, | \, y_i - \hat{y}(t_i) \, | \, |^2 \tag{3}$$

where $\hat{y}(t)$ is the predicted state trajectory produced by the ODE or SDE solver, and $| \, | \cdot | \, |$ denotes the standard Euclidean norm.

2.4. Backpropagation

A differentiable ODE solver computes the forward trajectory, and gradients are propagated through the integration process. To derive the adjoint equation from first principles, consider a small time step ε. Using a first-order Taylor expansion:

$$y(t + \varepsilon) = y(t) + \varepsilon f(y(t), t, \theta) + O(\varepsilon^2) \tag{4}$$

[2] PTB-XL version 1.0.3 is available from PhysioNet at https://physionet.org/content/ptb-xl/1.0.3/.

By the chain rule, the sensitivity of the loss $\mathcal{L}$ with respect to the state at time t, defined as the adjoint state $a(t) = \partial\mathcal{L}/\partial y(t)$, is:

$$a(t) = \frac{\partial \mathcal{L}}{\partial y(t)} = \frac{\partial \mathcal{L}}{\partial y(t+\varepsilon)} \frac{\partial y(t+\varepsilon)}{\partial y(t)} \tag{5}$$

Substituting the expansion from (4) into (5):

$$a(t) = a(t+\varepsilon)\left(I + \varepsilon \frac{\partial f(y(t), t, \theta)}{\partial y(t)}\right) \tag{6}$$

where I is the identity matrix. Rearranging and taking the limit $\varepsilon \to 0$ yields the adjoint ODE:

$$\frac{da(t)}{dt} = -a(t)^T \frac{\partial f(y(t), t, \theta)}{\partial y} \tag{7}$$

Here, $a(t)^T$ denotes the transpose of the adjoint state vector (a row vector representing how sensitive the loss is to perturbations in each state dimension), and $\frac{\partial f(y(t),t,\theta)}{\partial y}$ is the Jacobian matrix representing the partial derivatives of the vector field f with respect to the state y. Solving this linear differential equation backwards in time enables gradient computation with constant memory cost, instead of storing every intermediate step in memory to apply the chain rule in the backward pass.

3. Implementation

NODEFit is implemented as an open-source Python package built on top of the PyTorch [12] ecosystem. It leverages specialized libraries to handle the numerical integration and gradient computation required for training Neural ODEs and SDEs. By abstracting these complexities, NODEFit makes it remarkably easy to fit complex time-series data to governing differential equations. All plots in this paper were generated using Matplotlib [13].

3.1. Core Dependencies

The efficiency and scalability of NODEFit rely on two primary libraries: `torchdiffeq` and `torchsde`.

3.1.1. *torchdiffeq*

For Ordinary Differential Equations, NODEFit utilizes `torchdiffeq` [3]. This library provides a suite of differentiable ODE solvers. A critical feature of `torchdiffeq` is its support for the **adjoint sensitivity method**. Unlike standard backpropagation through the solver's internal operations (which has a memory cost that scales with the number of solver steps), the adjoint method allows for gradient computation with constant memory cost by solving an augmented ODE backwards in time. The adjoint state $a(t) = \partial\mathcal{L}/\partial y(t)$ follows the dynamics:

$$\frac{da(t)}{dt} = -a(t)^T \frac{\partial f(y(t), t, \theta)}{\partial y} \tag{8}$$

The gradient with respect to the model parameters θ is then computed by integrating backwards from the final time t_1 to the initial time t_0:

$$\frac{d\mathcal{L}}{d\theta} = -\int_{t_1}^{t_0} a(t)^T \frac{\partial f(y(t), t, \theta)}{\partial \theta} dt \tag{9}$$

where $\frac{\partial f(y(t),t,\theta)}{\partial \theta}$ represents the partial derivative (Jacobian) of the vector field with respect to the neural network parameters θ.

This enables the training of complex models on large time-series datasets that would otherwise be computationally prohibitive. The memory efficiency stems from the fact that the adjoint method does not require storing intermediate states $y(t)$ from the forward pass.

Instead, the original ODE is solved backwards in time alongside the adjoint equation, reconstructing the state $y(t)$ on the fly. This reduces the memory complexity from $O(N)$, where N is the number of solver steps, to $O(1)$ relative to the trajectory length, at the possible cost of sensitivity to errors in the backpropagation.

3.1.2. *torchsde*

For stochastic systems, NODEFit integrates `torchsde` [4]. Stochastic Differential Equations present unique challenges, particularly in ensuring consistent Brownian motion across multiple steps and handling the nuances of stochastic calculus.

Similar to the deterministic case, the stochastic adjoint sensitivity method avoids storing the full trajectory, enabling gradient computation with constant memory cost. However, SDEs require consistent noise across both forward and backward passes. `torchsde` achieves this through a **Virtual Brownian Tree**, which allows for the exact reconstruction of the Brownian motion W_t at any time point using a fixed seed. By reconstructing both the state and the noise during the backward pass, the memory cost remains constant even for complex stochastic trajectories. For the complete first-principles derivation of the stochastic adjoint and the associated stochastic calculus, see the Appendix.

3.1.3. *Memory Efficiency Trade-offs*

The primary advantage of the adjoint methods leveraged by NODEFit is the reduction in memory overhead. The mathematical adjoint formulation (integrating the adjoint state backwards in time) and the Virtual Brownian Tree noise reconstruction are implemented within the underlying PyTorch differential equation ecosystem (`torchdiffeq` and `torchsde`). The following table summarizes the theoretical scaling and computational trade-offs between standard naive backpropagation and the adjoint-based sensitivity methods in the PyTorch libraries `torchdiffeq` and `torchsde`.

Table 1. *Comparison of memory efficiency and computational trade-offs between naive backpropagation and the adjoint sensitivity method in the PyTorch libraries* `torchdiffeq` *and* `torchsde`*.*

Feature	Naive Backprop	Adjoint Method (PyTorch libraries)
Intermediate States	Stored in memory	Reconstructed on the fly
Memory Scaling	$O(N)$ (Linear with steps)	$O(1)$ (Constant with steps)
Noise (SDEs)	Must be stored for every step	Regenerated via Virtual Brownian Tree
Computational Trade-off	Faster (no reconstruction)	Slower (requires solving backwards)
Error Sensitivity	Standard backpropagation	Possible sensitivity to errors in backpropagation

While the $O(1)$ asymptotic memory scaling is delivered by the underlying PyTorch solvers, NODEFit's contribution lies in wrapping these complex mathematical engines into a single training API, coordinated drift and diffusion updates, observation batching, and the `FastNeuralSDE` execution path used for long series and large cohorts.

3.2. Performance Optimizations

To handle larger datasets and more complex trajectories, NODEFit includes an optimized implementation (`FastNeuralSDE`) that inherits from the base `NeuralSDE` and `SDE` classes. Key optimizations include:

1. **In-place tensor allocation:** Using `y.new_full` for state-time concatenation in drift/diffusion functions (f and g), avoiding redundant tensor creation and device transfers during solver evaluations.
2. **Single-precision arithmetic:** Operating consistently in `float32` rather than `float64`.
3. **Solver configuration & loop efficiency:** Enforcing a fixed time step `dt = 0.1` for Euler integration and pre-allocating repeated target batch tensors to avoid redundant allocations during loss computation.

Benchmarking against the base implementation on synthetic trajectories demonstrates an empirical speedup of $138\times$ (reducing per-epoch training time from 1.84 s down to 0.013 s on the reference CPU), bringing 1,000 epochs of Neural SDE training down to under 40 seconds. While base `NeuralSDE` serves as a simple double-precision reference for small prototypes, `FastNeuralSDE` is the recommended default for production workflows, longer series, and large cohort training.

3.3. Reproducibility and wall-clock timings

The example scripts were run once on a 14-inch 2021 MacBook Pro with an Apple M1 Pro chip and 16 GB of memory, running macOS Tahoe 26.3.1. The wall-clock times include model initialization, training, extrapolation, and plot generation, and are provided as reproducibility information rather than as a benchmark. They can vary with software versions and system load.

Table 2. *Wall-clock times for the example scripts on the reference laptop.*

Example script	Training epochs	Wall-clock time
`simple_ode_only.py`	1000	50 s
`simple_sde_only.py`	1000	36 s
`sde_ode_trajectory.py`	500	32 s

3.4. Installation

NODEFit can be installed via `pip`:

```
pip install nodefit
```

- **Python and PyTorch Versions**: NODEFit requires Python ≥ 3.8 and PyTorch ≥ 1.12 (compatible with PyTorch 2.x). Core dependencies, including `torchdiffeq`, `torchsde`, `numpy` and `matplotlib`, are installed automatically.
- **Hardware Requirements**: A dedicated GPU is optional. Because the neural vector fields and diffusion networks for typical continuous-time dynamical systems are relatively compact (often 1–3 shallow layers with modest hidden dimensions), CPU-only usage is fully supported and fast (e.g., all benchmark models in this paper train in under a minute on a standard laptop CPU). When available, GPU acceleration (via CUDA or Apple Silicon MPS) can be utilized seamlessly by placing tensors and modules on the target device using standard PyTorch semantics.

3.5. Sample Code

The following example fits a Neural SDE to a noisy 2D trajectory. Times and states are ordinary NumPy arrays, with the user supplying the two networks and NODEFit handling the batching, the training loop, and extrapolation. A domain scientist does not need to write a differentiable solver, an adjoint backward pass, or separate parameter groups for drift and diffusion.

```python
import numpy as np
import torch
import torch.nn as nn
from nodefit.neural_sde import NeuralSDE

# Set seed for reproducibility
torch.manual_seed(0)
np.random.seed(0)

# Generate noisy synthetic time-series data (2D)
t = np.linspace(0, 5, 50)
y1 = 1.0 + 2.2 * (1 - np.exp(-0.5 * t)) + np.random.normal(0, 0.1, 50)
y2 = 1.0 + 0.6 * (1 - np.exp(-0.5 * t)) + np.random.normal(0, 0.1, 50)
data = np.stack([y1, y2], axis=1)

# Define the drift network (f_theta)
# Input: (t, y), so input_dim = 1 + 2 = 3
drift_nn = nn.Sequential(
    nn.Linear(3, 20),
    nn.Tanh(),
    nn.Linear(20, 2)
).double()

# Define the diffusion network (g_theta)
diffusion_nn = nn.Sequential(
    nn.Linear(3, 20),
    nn.Tanh(),
    nn.Linear(20, 2)
).double()

# Initialize and train the SDE model: fitting data is as simple as passing the observations
sde = NeuralSDE(drift_nn, diffusion_nn, t, data, batch_size=20)
sde.train(num_epochs=500, print_every=100)

# Extrapolate to future time points
extrapolated = sde.extrapolate(tf=8, npts=40)
```

4. Results

We evaluated NODEFit's PyTorch-based Neural ODEs and SDEs on both deterministic and stochastic benchmarks drawn from the motivating scenario in the Introduction: coupled states approaching saturation with optional noise. The models were trained using the Adam optimizer with default learning rates. In each case, the goal is not merely to interpolate scattered points but to recover a coherent evolution law that extrapolates beyond $t = 5$. Figure 1 through Figure 4 illustrate settings where conventional curve fitting would require the correct functional template *a priori*, and where treating noise as homoscedastic regression error would misrepresent uncertainty during the forecast.

4.1. Neural ODE Results

For the deterministic case, we first fit the same training data with `scipy.optimize.curve_fit` [6], using a cubic polynomial ($y = a + bt + ct^2 + dt^3$) fitted independently to each state. The extra flexibility tracks the training window closely, but without a saturation mechanism the extrapolation past $t = 5$ inflects upward rather than leveling off (Figure 1, dashed curves). We then trained a Neural ODE for 1000 epochs on the same 2D system. The learned flow matches the saturating trajectories and continues smoothly toward steady state beyond the training window, without specifying the functional form in advance.

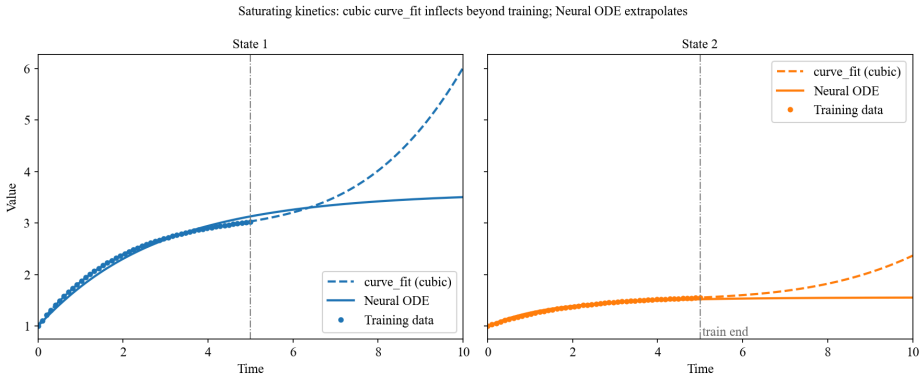


Figure 1. *Baseline failure and Neural ODE success on coupled saturating kinetics. Dashed curves show* `curve_fit` *with a cubic polynomial template; solid curves show the Neural ODE integrated through* $t = 10$*. The vertical dotted line marks the end of training data* ($t = 5$)*.*

4.2. Classical reconstruction: Lotka–Volterra dynamics

To demonstrate an application with a known scientific interpretation, we reconstruct the Lotka–Volterra predator–prey model from partial observations. This model was developed independently by Lotka and Volterra to describe interacting populations [14]. With $x(t)$ denoting prey and $z(t)$ denoting predator population, the classical equations are

$$\begin{aligned} \frac{dx}{dt} &= \alpha x - \beta xz, \\ \frac{dz}{dt} &= \delta xz - \gamma z. \end{aligned} \tag{10}$$

The first term in each equation describes unconstrained population growth or decay, while the interaction terms describe predation. We generated a trajectory with $(\alpha, \beta, \gamma, \delta) = (1.5, 1.0, 3.0, 1.0)$ and initial state $(x(0), z(0)) = (1.5, 1.0)$. Only observations up to $t = 6$ were supplied to a structured Neural ODE whose four positive coefficients were learned jointly with the trajectory. The learned values were $(1.52, 1.01, 2.94, 0.98)$, close to the generating system. As shown in Figure 2, the model reconstructs both the oscillatory population dynamics and the closed phase portrait, then extrapolates the cycle to $t = 12$. This example illustrates how NODEFit can recover an interpretable continuous-time law from incomplete observations, and in applications where the governing terms are not known, the same interface can instead use an unconstrained neural drift.

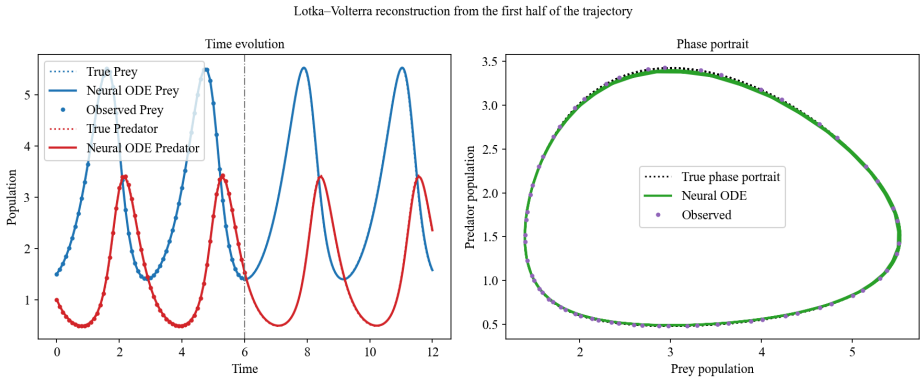


Figure 2. *Lotka–Volterra reconstruction from observations through* $t = 6$*. Dotted curves show the classical system used to generate the data, solid curves show the learned structured Neural ODE, and points show the observations.*

4.3. Neural SDE Results

In the stochastic case, we trained a Neural SDE for 300 epochs. The diffusion network learns to capture the noise characteristics of the data. Figure 3 illustrates the mean prediction along with the standard deviation (shaded regions) across multiple trajectories. The model successfully captures the underlying trend while quantifying the uncertainty, which increases during extrapolation.

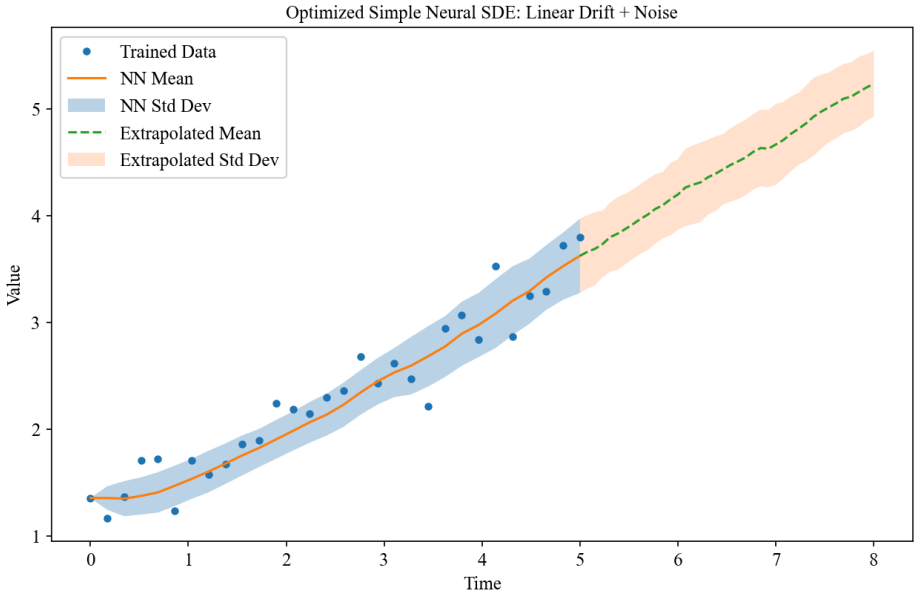


Figure 3. *Neural SDE fit and extrapolation. The shaded regions represent the standard deviation across 10 trajectories, capturing the learned diffusion process.*

4.4. Fitting Complex Dynamics

We also tested NODEFit's PyTorch-based Neural SDE on a more complex 2D system with noise. The underlying theoretical trajectories for this system are governed by the following equations:

$$\begin{aligned} y_1(t) &= 1.0 + 2.2(1 - e^{-0.5t}) \\ y_2(t) &= 1.0 + 0.6(1 - e^{-0.5t}) \end{aligned} \tag{11}$$

The model was tasked with learning these underlying dynamics and providing robust extrapolations. As shown in Figure 4, the Neural SDE successfully recovers the mean trajectory, closely matching the underlying theoretical dynamics even in the presence of noise. The comparison between the theoretical mean and the predicted mean demonstrates the model's ability to filter out stochastic fluctuations and capture the true governing laws.

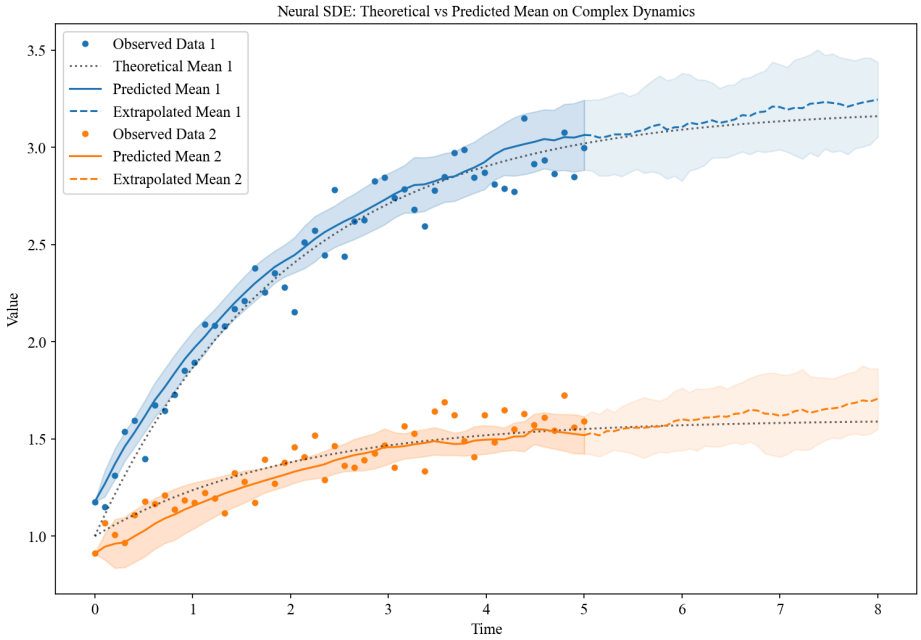


Figure 4. *Neural SDE fit on a complex 2D trajectory. The dotted black line represents the underlying theoretical trajectory, while the solid lines and shaded regions show the predicted mean and uncertainty. The model captures the multi-dimensional dynamics and provides reliable extrapolations.*

4.5. Clinical Case Study: Stochastic Heart Rate Dynamics in Atrial Fibrillation

To evaluate NODEFit's PyTorch-based Neural SDE on a clinical research dataset, we analyzed an electrocardiogram (ECG) recording from the open-source PTB-XL database [15], [16]. In patients diagnosed with cardiac arrhythmias such as atrial fibrillation (AFIB), the sinoatrial node fires irregularly, producing stochastic fluctuations in the beat-to-beat interval (RR) and instantaneous heart rate alongside baseline sensor and environmental noise.

An ECG measures the cardiac electrical conduction cycle, characterized by the P-Q-R-S-T sequence: the P wave reflects atrial depolarization, the QRS complex corresponds to rapid ventricular depolarization, and the T wave represents ventricular repolarization[3].

Clinical interpretation typically pairs a limb rhythm lead with a left-precordial chest lead, so we evaluated Lead II and Lead V5 of PTB-XL record #16834 (Figure 5). Because both leads were recorded simultaneously from the same patient on the same multi-channel device, the physiological heartbeats occur synchronously, and R-peak detection recovers the identical 18 irregular beats across both channels. However, Lead V5 is positioned directly over the left ventricle, yielding taller QRS voltage amplitudes (0.82 mV peak vs. 0.56 mV on Lead II) and a cleaner signal-to-noise ratio than Lead II.

[3]For standard clinical definitions of the ECG waveform, cardiac conduction cycles, and lead placements, see Goldberger et al. [17].

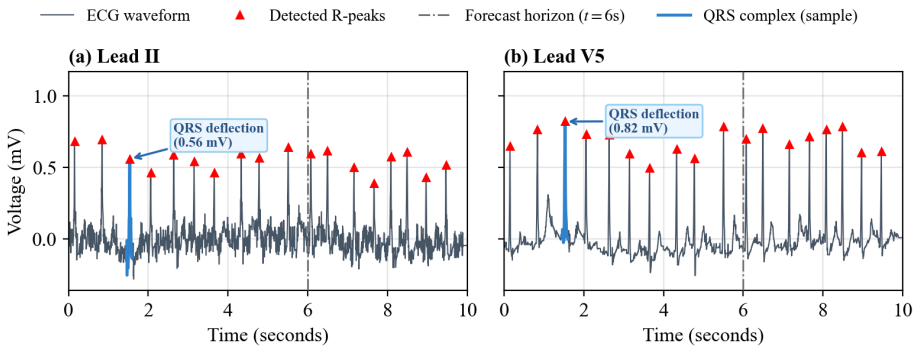


Figure 5. *Ten-second clinical ECG from PTB-XL record #16834 (500 Hz). (a) Lead II. (b) Lead V5. Red triangles mark detected R-peaks; blue highlighted segments and arrows indicate a representative QRS deflection (*0.56 mV *on Lead II vs.* 0.82 mV *on Lead V5); the vertical dash-dotted line at* $t = 6$ s *marks the forecast horizon.*

Traditional discrete-time autoregressive models struggle with continuous physiological processes because observation intervals in biological time series vary continuously. In contrast, continuous-time Neural SDEs naturally operate over arbitrary time coordinates. Instantaneous heart rate is derived from RR intervals, so the two leads describe the same arrhythmia when peaks are detected reliably. For each lead we interpolated a dense continuous heart-rate trajectory, trained a compact Neural SDE on the first 6 seconds ($t \leq 6$ s, $N = 57$), and held out the remainder ($t > 6$ s, $N = 33$), as shown in Figure 6. As a deterministic baseline, `scipy.optimize.curve_fit` with a cubic polynomial was fitted over the same training window.

The cubic polynomial overfits local curvature during training and inflects rigidly upward past $t = 6$ s without bounding physiological extremes or quantifying uncertainty. NODEFit's Neural SDE, parameterized with bounded multi-harmonic phase coordinates ($K = 3$ harmonic orders spanning the underlying ~ 0.35 Hz autonomic cycle) and state-dependent drift, reconstructs the multi-cycle respiratory oscillation throughout the training window without unbounded extrapolation blowup. The diffusion network supplies a stochastic envelope of about $\pm 1\sigma \approx 9$ BPM and $\pm 2\sigma \approx 18$ BPM on both leads. Held-out RMSE is 24.4 BPM (Lead II) and 25.0 BPM (Lead V5) for the SDE mean, versus 29.5 BPM and 26.5 BPM for the cubic baseline. About 45-48% of held-out samples fall inside the $\pm 1\sigma$ band and 61% inside $\pm 2\sigma$, with the large late excursion near $t \approx 8$ s sitting at the edge of the envelope. The clinically useful contrast is the same on both leads, with the SDE supplying an oscillatory forecast and a quantified uncertainty band, whereas a polynomial template cannot.

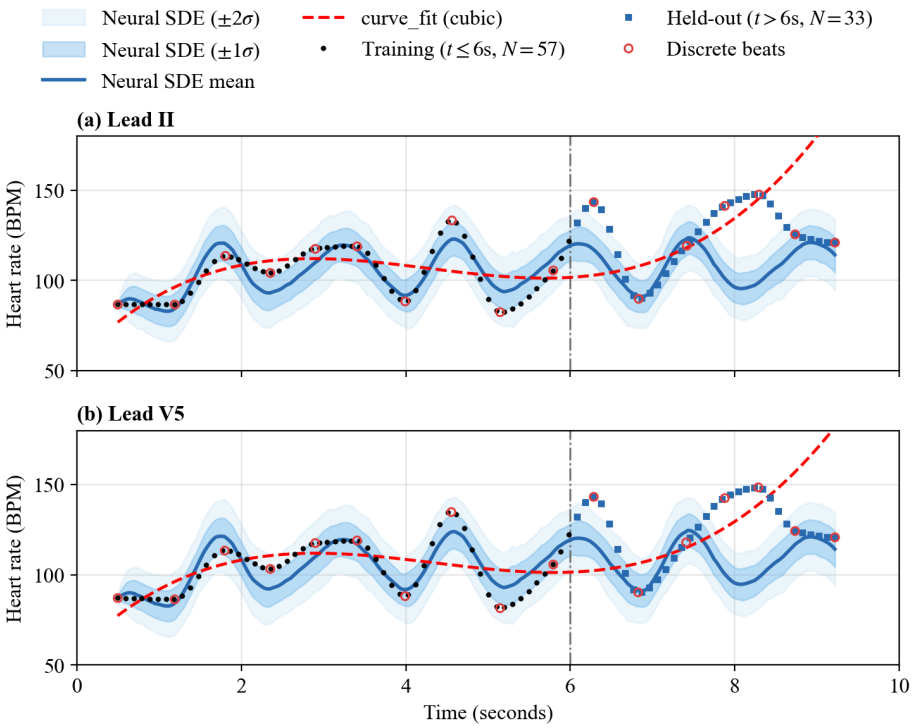


Figure 6. *Continuous-time instantaneous heart rate from record #16834. (a) Lead II. (b) Lead V5. Black dots are training observations ($t \leq 6$ s, $N = 57$), blue squares are held-out future observations ($t > 6$ s, $N = 33$), open red circles are discrete beats, the red dashed curve is the* `curve_fit` *cubic baseline, and the solid blue curve with shaded bands is the Neural SDE mean and diffusion ($\pm 1\sigma$, $\pm 2\sigma$).*

4.6. Large-Scale Cohort Benchmark on PTB-XL

To assess whether these continuous-time stochastic dynamics generalize across diverse clinical populations, we scaled the evaluation to a cohort sampled from 2,000 unique patient records in PTB-XL. Because limb lead II and precordial lead V5 exhibit distinct signal morphologies and noise characteristics, we analyzed both leads independently across the cohort. Approximately 55% of the cohort comprises patients with normal sinus rhythm (`NORM`), while 45% presents diagnostic abnormalities including atrial fibrillation, conduction blocks, and myocardial infarctions.

Out of the 2,000 patient records, 1,997 records successfully yielded valid continuous heart-rate trajectories across one or both leads, resulting in 3,948 individual lead samples (1,968 on Lead II and 1,980 on Lead V5). Exactly 3 patient records were discarded by the preprocessing pipeline due to severe physiological bradycardia and conduction blocks where 10-second recordings contained insufficient beat occurrences across both leads. Furthermore, because R-peak detection and physiological RR-interval criteria (the beat-to-beat time between successive R-peaks; $0.25 \text{ s} \leq RR \leq 2.5 \text{ s}$) were evaluated independently on each lead, 29 samples from Lead II and 17 samples from Lead V5 did not meet the beat-count threshold for spline interpolation, yielding 3,948 total usable samples. Each valid lead trajectory was extracted and fitted with the same NODEFit protocol ($t \leq 6$ s training, $t > 6$ s held-out forecasting), reusing one drift–diffusion architecture rather than a per-record solver script. Table 3 summarizes the aggregate forecasting error and empirical uncertainty coverage across all 3,948 samples.

Table 3. *Forecasting performance and uncertainty coverage across 3,948 lead samples from 1,997 unique PTB-XL patients, stratified by lead and diagnostic class.*

Lead	Cohort Stratum	N Samples	Neural SDE RMSE (BPM)	Cubic `curve_fit` RMSE (BPM)	Within 1σ (%)	Within 2σ (%)	Mean Diffusion $\pm 1\sigma$ (BPM)
Lead II	All	1968	10.43 ± 15.62	51.78 ± 111.42	88.5%	94.3%	16.4 BPM
Lead II	Normal	1083	8.02 ± 12.56	35.99 ± 86.57	93.4%	97.1%	16.2 BPM
Lead II	Arrhythmia / Abnormal	885	13.38 ± 18.28	71.12 ± 133.30	82.6%	91.0%	16.8 BPM
Lead V5	All	1980	8.02 ± 11.22	35.58 ± 70.21	91.9%	96.7%	16.5 BPM
Lead V5	Normal	1083	6.34 ± 8.43	26.59 ± 52.89	95.9%	98.5%	16.2 BPM
Lead V5	Arrhythmia / Abnormal	897	10.06 ± 13.58	46.44 ± 85.40	87.0%	94.6%	16.8 BPM

Across all cohort subsets, NODEFit's Neural SDE significantly outperforms the standard polynomial `curve_fit` baseline, reducing held-out forecast RMSE by a factor of 4.4 to 5.0 (10.43 BPM vs. 51.78 BPM on Lead II; 8.02 BPM vs. 35.58 BPM on Lead V5). The performance advantage is particularly pronounced in abnormal and arrhythmic patients, where the deterministic polynomial rapidly diverges on non-stationary trajectories (71.12 BPM error), while the Neural SDE constrains drift and maintains bounded error (13.38 BPM).

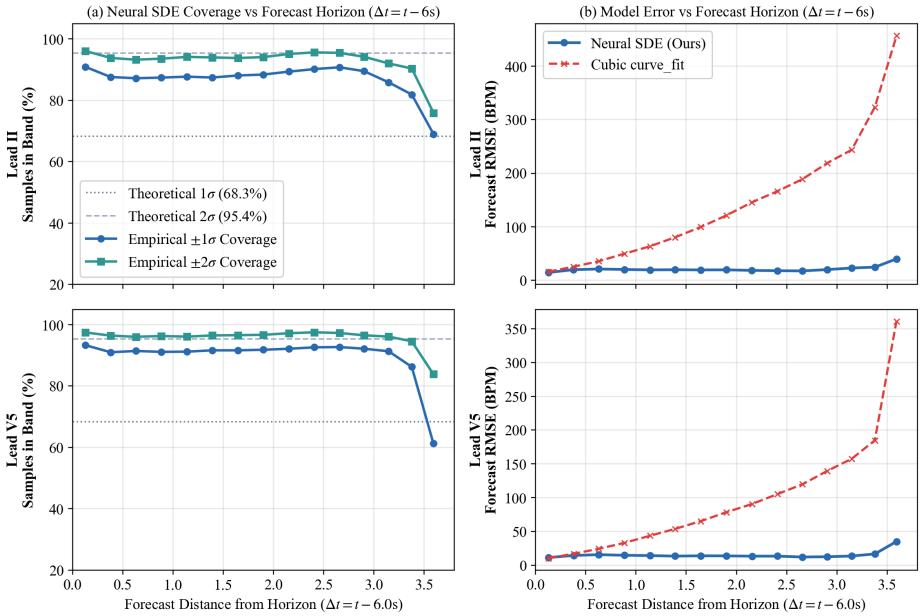


Figure 7. *Cohort-wide forecasting reliability as a function of temporal distance from the forecast horizon ($\Delta t = t - 6$ s) across 3,948 lead samples (1,968 on Lead II, 1,980 on Lead V5) from 1,997 PTB-XL patients. Top row: Lead II; bottom row: Lead V5. (a) Empirical percentage of held-out observations encapsulated by the learned Neural SDE $\pm 1\sigma$ and $\pm 2\sigma$ diffusion envelopes compared against theoretical Gaussian coverage limits (68.3% and 95.4%). (b) Out-of-sample RMSE across forecast distance, demonstrating stable error bounds for NODEFit versus cubic polynomial runaway.*

To examine how predictive uncertainty evolves into the unobserved future, Figure 7 tracks model performance as a function of distance $\Delta t = t - 6$ s from the forecast horizon:

1. **Uncertainty Calibration:** As shown in Figure 7 (column a), the learned diffusion network produces well-calibrated confidence bands throughout the forecast window. Across both Lead II and Lead V5, empirical $\pm 2\sigma$ coverage remains between 94% and 97%, closely aligning with the theoretical 95.4% two-standard-deviation Gaussian boundary.
2. **Extrapolation Stability:** As shown in Figure 7 (column b), standard polynomial regression exhibits severe runaway divergence beyond $\Delta t > 1.5$ s, with average forecast errors exceeding 350-400 BPM near the end of the 10-second recording. In contrast, the Neural SDE's learned vector field maintains flat, physiologically bounded error profiles throughout the entire extrapolation interval.

These cohort-wide results confirm that continuous-time Neural SDEs provide robust trajectory modeling and calibrated uncertainty quantification across large, heterogeneous clinical datasets.

4.7. Future work with the ECG dataset

In our future work, we will test our software against additional benchmarks that are better suited for ECG signal comparisons. Examples of functions include: sines, cosines, dying exponentials, constants, negative powers, etc., as suggested by a reviewer.

5. Conclusion

NODEFit offers a user-friendly and time-saving tool for fitting continuous-time models to time-series data. By leveraging PyTorch's Neural ODE and SDE libraries, it enables the discovery of governing laws from observations, bridging the gap between machine learning and physical modeling.

For practitioners deciding whether to use NODEFit's PyTorch-based Neural ODEs and SDEs, the central question is whether the data plausibly arise from a smooth, Markovian continuous-time process. If polynomial regression, splines, `scipy.optimize.curve_fit` with a hand-chosen template, or PySINDy with a library that already contains the true terms produces stable fits and credible extrapolations, a Neural ODE may be unnecessary. PySINDy is the better choice when an interpretable sparse equation in a known basis is the goal. Consider NODEFit when those tools leave systematic residuals, extrapolations diverge from physical expectations, or the functional form of the dynamics is unknown, including stochastic processes where a diffusion network is needed. Template-based fits must guess a closed-form expression or a candidate library, whereas a Neural ODE learns a single vector field coupling all states.

When observations are noisy, ask whether the noise reflects measurement error alone or variability intrinsic to the process. Ordinary least squares and deterministic Neural ODEs treat scatter as something to be averaged out. If uncertainty grows with state magnitude or if extrapolated forecasts should carry confidence intervals, a Neural SDE is the more appropriate model. The diffusion network learns state-dependent noise alongside the drift.

The reason to use NODEFit rather than those PyTorch libraries directly is the wrapping workflow. NODEFit accepts NumPy times and states, standardizes the training loop, batches observations, and keeps drift and diffusion on coordinated optimizers so the user does not maintain those parameter groups by hand. The API is built for the actions that dominate applied fitting, such as swapping a network's width or depth, retraining, and comparing extrapolations without changing solver settings. That matters most for large batch studies with extensive hyperparameter searches, such as the PTB-XL cohort in Section 15, which applied one protocol to nearly 2,000 records.

NODEFit is intended for scientists who need a Neural ODE or SDE fit without assembling adjoint code, Brownian-tree bookkeeping, and device transfers themselves. `FastNeuralSDE` is the NODEFit-specific execution path that reduces that cost further, with a measured $138\times$ speedup relative to the package's own reference `NeuralSDE` class.

6. GenAI Policy

Portions of this work were assisted using a generative AI tool, Cursor. The tool was used for drafting text, refining language, or generating code suggestions. All outputs were reviewed, verified, and revised by the author, who takes full responsibility for the accuracy and integrity of the final content.

7. Appendix: Stochastic Calculus and the Adjoint Method

The derivation of the stochastic adjoint sensitivity method relies on the choice of stochastic integral. This appendix provides the necessary background on the Itô and Stratonovich formulations.

7.1. Martingales

A stochastic process M_t is a **martingale** if its expected future value, given all the information available up to the current time t, is exactly its current value:

$$E[M_s \mid \text{information up to time } t] = M_t \tag{12}$$

Intuitively, a martingale represents a "fair game" where there is no systematic tendency to increase or decrease. This property is fundamental for ensuring that a stochastic model does not have an unintended "hidden" drift.

7.2. Itô vs. Stratonovich Integrals

For a stochastic process $y(t)$ governed by a diffusion term $g(y,t)$, the integral with respect to Brownian motion W_t can be defined in two primary ways depending on the evaluation point within a time interval $[t, t+\Delta t]$:

1. **Itô Integral** (denoted $g\,dW_t$): Evaluates the integrand at the **left endpoint** t. Because the integrand is evaluated before the noise increment dW_t occurs, they are independent. Since Brownian increments have zero mean, the expected change is zero, making the Itô integral a **martingale**. This makes it the standard choice for modeling physical systems where noise should not introduce systematic drift. However, it does not follow the standard chain rule of calculus.
2. **Stratonovich Integral** (denoted $g \circ dW_t$): Evaluates the integrand at the **midpoint** $t + \Delta t/2$. This creates a correlation between the integrand and the noise, which introduces a drift and causes the integral to **lose the martingale property**. However, its primary advantage is that it **obeys the standard rules of calculus** (chain rule, product rule), which simplifies the derivation of adjoint equations.

7.3. Stochastic Adjoint Derivation

To derive the stochastic adjoint from first principles, we consider the SDE in Stratonovich form (denoted by the $\circ$ operator). The Stratonovich integral evaluates the integrand at the midpoint of the interval, $g(y,t) \circ dW_t \approx g\big(y_{t+\Delta t/2}, t + \Delta t/2\big)\Delta W_t$. This choice ensures that the SDE obeys the standard rules of calculus:

$$dy = f(y,t,\theta)dt + g(y,t,\theta) \circ dW_t \tag{13}$$

Consider a small time step Δt. The state update is approximately:

$$y(t+\Delta t) \approx y(t) + f(y(t),t,\theta)\Delta t + g(y(t),t,\theta)\Delta W_t \tag{14}$$

Following the same chain rule logic as in (5), the sensitivity of the loss with respect to the state at time t is:

$$a(t) = \left(\frac{\partial y(t+\Delta t)}{\partial y(t)}\right)^T a(t+\Delta t) \tag{15}$$

Substituting the derivative of (14):

$$a(t) \approx \left(I + \frac{\partial f}{\partial y}^T \Delta t + \sum_j \frac{\partial g_j}{\partial y}^T \Delta W_{t,j}\right) a(t+\Delta t) \tag{16}$$

In the limit $\Delta t \to 0$, this yields the adjoint SDE in Stratonovich form:

$$da(t) = -\left(\frac{\partial f}{\partial y}\right)^T a(t)dt - \sum_j \left(\frac{\partial g_j}{\partial y}\right)^T a(t) \circ dW_{t,j} \tag{17}$$

When converted back to Itô form for numerical stability and implementation, this introduces the **Stratonovich-to-Itô correction** term:

$$da(t) = -\left[a(t)\frac{\partial f}{\partial y} - \sum_j \left(a(t)\frac{\partial g_j}{\partial y}\right)\frac{\partial g_j}{\partial y}\right]dt - \sum_j \left(a(t)\frac{\partial g_j}{\partial y}\right)dW_t \tag{18}$$

where g_j are the columns of the diffusion matrix g.

7.4. Stratonovich-to-Itô Conversion and Correction

The relationship between a Stratonovich SDE ($dy = f_s dt + g \circ dW_t$) and an Itô SDE ($dy = f_i dt + g dW_t$) is given by the conversion formula:

$$f_i(y,t) = f_s(y,t) + \frac{1}{2}\sum_j \left(\frac{\partial g_j(y,t)}{\partial y}\right) g_j(y,t) \tag{19}$$

where g_j are the columns of the diffusion matrix. The second term is the **Stratonovich-to-Itô correction**. In `torchsde`, derivations are performed in the Stratonovich framework to leverage standard calculus, while numerical solvers often operate in the Itô framework, requiring the explicit inclusion of this correction term in the drift dynamics.

It is important to note that for the adjoint SDE (Equation (18)), the $1/2$ factor is absent. This is because the correction term for the adjoint state $a(t)$ arises from the interaction between the forward state $y(t)$ and the adjoint variable. When converting the augmented system (y, a) to Itô form, the resulting drift correction for $a(t)$ consists of two identical terms from the Stratonovich expansion that sum to unity, effectively canceling the $1/2$ coefficient found in the standard forward conversion formula [4].

7.4.1. *Mathematical Derivation*

To see this mathematically from first principles, we use index notation where y_i and a_i denote the components of the state and adjoint (row) vectors.

1. General Midpoint Rule and Conversion: For any Stratonovich stochastic term $G(X,t) \circ dW$ with state vector $X(t)$, the integral over a small time step $[t, t+\Delta t]$ is evaluated at the midpoint $\bar{X} = X(t) + \frac{1}{2}\Delta X$:

$$G_{i,j}(\bar{X},t)\Delta W_j \approx \left[G_{i,j}(X(t),t) + \frac{1}{2}\sum_k \frac{\partial G_{i,j}}{\partial X_k}\Delta X_k\right]\Delta W_j \tag{20}$$

Substituting the leading-order stochastic increment $\Delta X_k \approx \sum_l G_{k,l}\Delta W_l$ and applying the Brownian quadratic variation ($\Delta W_l \Delta W_j = \delta_{lj}\Delta t$):

$$\sum_j G_{i,j}(X,t) \circ dW_j = \sum_j G_{i,j}(X,t) dW_j + \frac{1}{2} \sum_j \sum_k \frac{\partial G_{i,j}}{\partial X_k} G_{k,j} dt \tag{21}$$

This fundamental midpoint relation provides the Stratonovich-to-Itô drift correction for any state vector X.

2. Forward Stratonovich Drift: Applying this general relation directly to the forward state $X = y$ with diffusion matrix $g(y,t)$:

$$\sum_j g_{i,j}(y,t) \circ dW_j = \sum_j g_{i,j}(y,t) dW_j + \frac{1}{2} \sum_j \sum_k \frac{\partial g_{i,j}}{\partial y_k} g_{k,j} dt \tag{22}$$

Equating the total drift of the Stratonovich SDE $dy_i = \tilde{f}_i dt + \sum_j g_{i,j} \circ dW_j$ to that of the forward Itô SDE $dy_i = f_i dt + \sum_j g_{i,j} dW_j$ yields the equivalent forward Stratonovich drift $\tilde{f}_i$:

$$\tilde{f}_i = f_i - \frac{1}{2} \sum_j \sum_k \frac{\partial g_{i,j}}{\partial y_k} g_{k,j} \tag{23}$$

3. Adjoint Itô Drift Correction: From Equation (17), the adjoint Stratonovich SDE is:

$$da_i = -\sum_k a_k \frac{\partial \tilde{f}_k}{\partial y_i} dt - \sum_j \sum_k a_k \frac{\partial g_{k,j}}{\partial y_i} \circ dW_j \tag{24}$$

Here, we identify the adjoint diffusion term $\sigma^{(a)}_{i,j}(y,a) = -\sum_k a_k \frac{\partial g_{k,j}}{\partial y_i}$. Because $\sigma^{(a)}_{i,j}$ depends simultaneously on both the forward state y and the adjoint state a, the augmented state vector is $X = [y, a]$.

Applying the general midpoint correction formula to this augmented system $X = [y, a]$ distributes the partial derivatives across both state variables, directly giving the adjoint drift correction C_{a_i}:

$$C_{a_i} = \frac{1}{2} \sum_j \left(\sum_k \frac{\partial \sigma^{(a)}_{i,j}}{\partial a_k} \sigma^{(a)}_{k,j} + \sum_k \frac{\partial \sigma^{(a)}_{i,j}}{\partial y_k} g_{k,j} \right) \tag{25}$$

where the first sum accounts for perturbations in the adjoint state $\Delta a_k \approx \sum_l \sigma^{(a)}_{k,l} \Delta W_l$, and the second sum accounts for perturbations in the forward state $\Delta y_k \approx \sum_l g_{k,l} \Delta W_l$.

4. Expansion and Cancellation: Evaluating each term in Equation (25) using $\sigma^{(a)}_{i,j} = -\sum_m a_m \frac{\partial g_{m,j}}{\partial y_i}$:

1. $\sum_k \frac{\partial \sigma^{(a)}_{i,j}}{\partial a_k} \sigma^{(a)}_{k,j} = \sum_k \left(-\frac{\partial g_{k,j}}{\partial y_i} \right) \left(-\sum_m a_m \frac{\partial g_{m,j}}{\partial y_k} \right) = \sum_m a_m \sum_k \frac{\partial g_{m,j}}{\partial y_k} \frac{\partial g_{k,j}}{\partial y_i}$
2. $\sum_k \frac{\partial \sigma^{(a)}_{i,j}}{\partial y_k} g_{k,j} = \sum_k \left(-\sum_m a_m \frac{\partial^2 g_{m,j}}{\partial y_k \partial y_i} \right) g_{k,j} = -\sum_m a_m \sum_k \frac{\partial^2 g_{m,j}}{\partial y_i \partial y_k} g_{k,j}$

Next, differentiating the Stratonovich drift $\tilde{f}_k$ from (23) with respect to y_i:

$$\frac{\partial \tilde{f}_k}{\partial y_i} = \frac{\partial f_k}{\partial y_i} - \frac{1}{2} \sum_j \sum_m \left(\frac{\partial^2 g_{k,j}}{\partial y_i \partial y_m} g_{m,j} + \frac{\partial g_{k,j}}{\partial y_m} \frac{\partial g_{m,j}}{\partial y_i} \right) \tag{26}$$

Combining everything into the total Itô drift for a_i, $\mu_{a_i} = -\sum_k a_k \frac{\partial \tilde{f}_k}{\partial y_i} + C_{a_i}$:

$$\begin{aligned} \mu_{a_i} = &-\sum_k a_k \left[\frac{\partial f_k}{\partial y_i} - \frac{1}{2} \sum_j \sum_m \left(\frac{\partial^2 g_{k,j}}{\partial y_i \partial y_m} g_{m,j} + \frac{\partial g_{k,j}}{\partial y_m} \frac{\partial g_{m,j}}{\partial y_i} \right) \right] \\ &+ \frac{1}{2} \sum_j \sum_m a_m \left(\sum_k \frac{\partial g_{m,j}}{\partial y_k} \frac{\partial g_{k,j}}{\partial y_i} - \sum_k \frac{\partial^2 g_{m,j}}{\partial y_i \partial y_k} g_{k,j} \right) \end{aligned} \tag{27}$$

The second-order derivative terms involving $\frac{\partial^2 g}{\partial y^2}$ cancel out exactly, while the products of first-derivative terms add up: $\frac{1}{2} + \frac{1}{2} = 1$. This yields the final Itô drift:

$$\mu_{a_i} = -\sum_k a_k \frac{\partial f_k}{\partial y_i} + \sum_j \sum_k a_k \sum_m \frac{\partial g_{k,j}}{\partial y_m} \frac{\partial g_{m,j}}{\partial y_i} \tag{28}$$

which is the component-wise form of Equation (18).

References

[1] R. T. Chen, Y. Rubanova, J. Bettencourt, and D. K. Duvenaud, "Neural Ordinary Differential Equations," *Advances in Neural Information Processing Systems*, vol. 31, 2018, doi: 10.48550/arXiv.1806.07366.

[2] Y. Rubanova, R. T. Chen, and D. K. Duvenaud, "Latent Ordinary Differential Equations for Irregularly-Sampled Time Series," *Advances in Neural Information Processing Systems*, vol. 32, 2019, doi: 10.48550/arXiv.1907.03907.

[3] R. T. Q. Chen, "torchdiffeq: Differentiable ODE Solvers with PyTorch." [Online]. Available: https://github.com/rtqichen/torchdiffeq

[4] X. Li, T.-K. L. Wong, R. T. Q. Chen, and D. Duvenaud, "Scalable Gradients for Stochastic Differential Equations," in *Proceedings of the 23rd International Conference on Artificial Intelligence and Statistics (AISTATS)*, 2020. doi: 10.48550/arXiv.2001.01328.

[5] P. Kidger, J. Foster, X. Li, H. Oberhauser, and T. Lyons, "Neural SDEs as infinite-dimensional GANs," in *Proceedings of the 38th International Conference on Machine Learning (ICML)*, 2021, pp. 5453–5463. doi: 10.48550/arXiv.2102.03657.

[6] P. Virtanen *et al.*, "SciPy 1.0: Fundamental Algorithms for Scientific Computing in Python," *Nature Methods*, vol. 17, pp. 261–272, 2020, doi: https://doi.org/10.1038/s41592-019-0686-2.

[7] G. E. P. Box, G. M. Jenkins, G. C. Reinsel, and G. M. Ljung, *Time Series Analysis: Forecasting and Control*, 5th ed. Wiley, 2015. doi: 10.1002/9781118619193.

[8] A. Vaswani *et al.*, "Attention Is All You Need," in *Advances in Neural Information Processing Systems*, 2017. doi: 10.48550/arXiv.1706.03762.

[9] I. Chevyrev and A. Kormilitzin, "A Primer on the Signature Method in Machine Learning." [Online]. Available: https://arxiv.org/abs/1603.03788

[10] B. M. de Silva, K. Champion, M. Quade, J.-C. Loiseau, J. N. Kutz, and S. L. Brunton, "PySINDy: A Python package for the sparse identification of nonlinear dynamical systems from data," *Journal of Open Source Software*, vol. 5, no. 49, p. 2104, 2020, doi: 10.21105/joss.02104.

[11] S. L. Brunton, J. L. Proctor, and J. N. Kutz, "Discovering governing equations from data by sparse identification of nonlinear dynamical systems," *Proceedings of the National Academy of Sciences*, vol. 113, no. 15, pp. 3932–3937, 2016, doi: 10.1073/pnas.1517384113.

[12] A. Paszke *et al.*, "PyTorch: An Imperative Style, High-Performance Deep Learning Library," in *Advances in Neural Information Processing Systems*, 2019, pp. 8024–8035. doi: 10.48550/arXiv.1912.01703.

[13] J. D. Hunter, "Matplotlib: A 2D graphics environment," *Computing in Science & Engineering*, vol. 9, no. 3, pp. 90–95, 2007, doi: 10.1109/MCSE.2007.55.

[14] V. Volterra, "Fluctuations in the Abundance of a Species Considered Mathematically," *Nature*, vol. 118, pp. 558–560, 1926, doi: 10.1038/118558a0.

[15] P. Wagner *et al.*, "PTB-XL, a large publicly available electrocardiography dataset," *Scientific Data*, vol. 7, no. 1, p. 154, 2020, doi: 10.1038/s41597-020-0495-6.

[16] A. L. Goldberger *et al.*, "PhysioBank, PhysioToolkit, and PhysioNet: Components of a new research resource for complex physiologic signals," *Circulation*, vol. 101, no. 23, pp. e215–e220, 2000, doi: 10.1161/01.CIR.101.23.e215.

[17] A. L. Goldberger, Z. D. Goldberger, and A. Shvilkin, *Goldberger's Clinical Electrocardiography: A Simplified Approach*, 9th ed. Elsevier, 2017. doi: 10.1016/c2014-0-03319-9.